

Modélisation Avancée

Matisse
kra
bts sio
Sommaire

Associations Ternaires	2
Objectif du TD :	2
1. Phase de Modélisation (MCD et MLD)	2
3. Requêtes SQL	3
Conclusion	5
Les associations réflexives	6
1. Contexte du projet	6
2. Modélisation Conceptuelle et Logique (Étapes 1 à 3)	6
3. Création et Alimentation de la Base de Données	6
4. Requêtage et Extraction de données	7
Conclusion et Bilan du TP	8
L'héritage	9
1.Objectif du TD :	9
2.création de l'heritage	9
conclusion	10
relation multiple	11
1.Objectifs du TP	11
2.Modélisation Conceptuelle (MCD)	11
3.base de donnée	12
Conclusion	13
la gestion des jointures	14
1.Objectifs du TP	14
2.Modélisation Conceptuelle (MCD)	14
3. Implémentation de la Base de Données	15
4. Travaux de Requêtage (SQL)	15
5. Conclusion	19
UNION & VUES	19
1. Présentation du Contexte	19
2. Problématique Technique	19
3. Compétences Mises en Œuvre	19
5. Schéma de Données (MCD/MLD)	20
6. conclusion	20
Gestion d'un Forum	21
1. Analyse et Modélisation (MCD) question 1 et 8	21
2. Mise en œuvre de la Base de Données (SQL) question 2 et 3, 9 et 10	21
3. Manipulation et Extraction de données question 4 a 7	22
Conclusion technique	23

Matisse kra
bts sio

Associations Ternaires

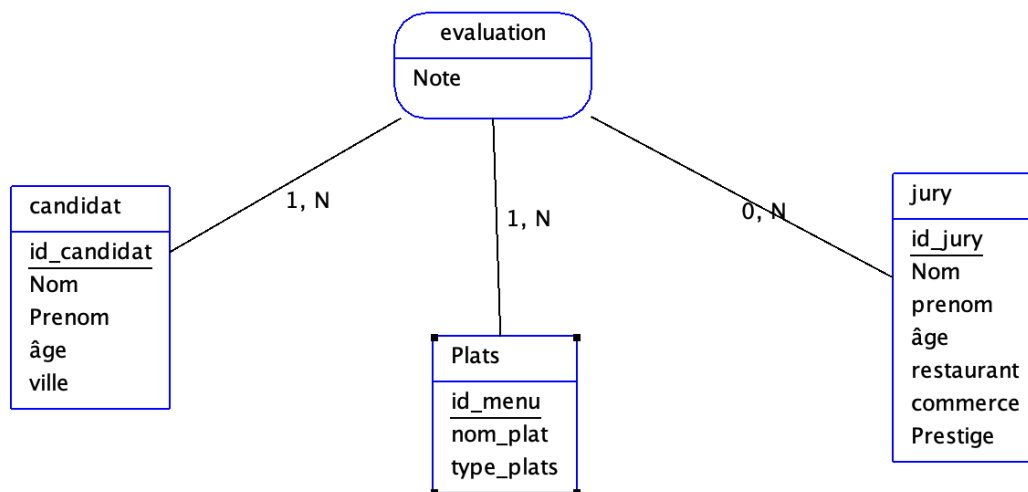
Objectif du TD :

Comprendre, modéliser et implémenter une base de données relationnelle à partir d'un cahier des charges impliquant des associations de dimension 3 (ternaires), puis interroger cette base avec des requêtes SQL complexes.

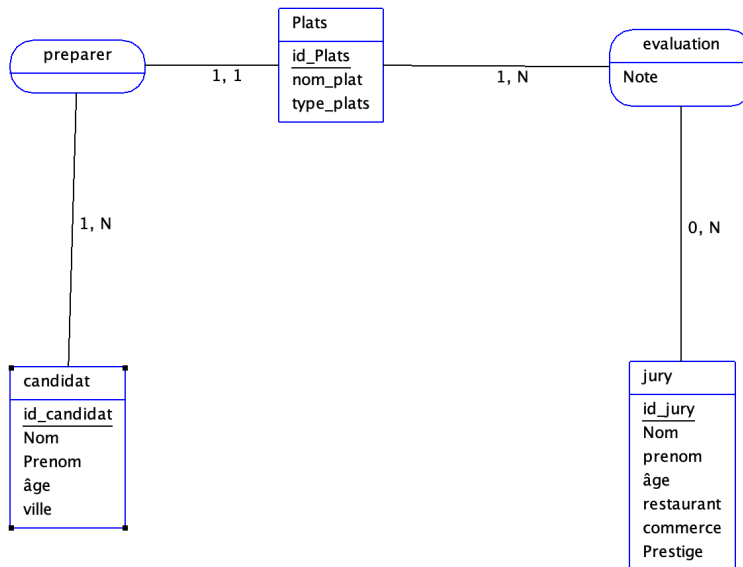
1. Phase de Modélisation (MCD et MLD)

La première étape s'est concentrée sur la conception de la base de données pour l'émission de cuisine « ChefauTop ».

- Gestion de l'association ternaire : Nous avons identifié que l'action d'évaluer un plat reliait trois entités distinctes : le Candidat, le Jury et le Plat et La propriété Note a été correctement placée au centre de cette association ternaire



- Simplification du Modèle Conceptuel de Données (MCD) : Nous avons ensuite restructuré le MCD en supprimant l'association ternaire au profit d'associations binaires. L'entité Plat est devenue le point central, reliée d'un côté au créateur (Candidat via 1,1 - 1,N) et de l'autre à l'évaluateur (Jury via 1,N - 0,N portant la note).



- Passage au Modèle Logique de Données (MLD) : Les règles de passage du modèle conceptuel au modèle logique ont été appliquées pour définir les clés étrangères et créer une table de liaison NOTER composée des clés primaires des entités reliées.

ancien mld

```

# Modèle créé le : Mon Mar 16 13:41:30 CET 2026
candidat (id_candidat_candidat, Nom_candidat, Prenom_candidat, âge_candidat, ville_candidat)
jury (id_jury_jury, Nom_jury, prenom_jury, âge_jury, restaurant_jury, commerce_jury, Prestige_jury)
Plats (id_menu_menu, nom_plat_Plats, type_plats_Plats)
evaluation (id_candidat_candidat, id_jury_jury, id_menu_menu, Note_evaluation)
    
```

nouveau mld

```

# Modèle créé le : Mon Mar 16 13:53:01 CET 2026
candidat (id_candidat_candidat, Nom_candidat, Prenom_candidat, âge_candidat, ville_candidat)
jury (id_jury_jury, Nom_jury, prenom_jury, âge_jury, restaurant_jury, commerce_jury, Prestige_jury)
Plats (id_Plats, nom_plat_Plats, type_plats_Plats, #id_candidat_candidat)
evaluation (id_jury_jury, id_Plats, Note_evaluation)
    
```

2. Phase d'Implémentation Physique (SQL DDL & DML)

À partir du MLD, le script de création de la base de données a été rédigé pour le SGBD (MariaDB/MySQL via XAMPP).

- Création des tables (DDL) : Les tables indépendantes PLAT, CANDIDAT et JURY ont été créées en premier avec leurs clés primaires respectives. La table associative NOTER a été créée avec ses contraintes de clés étrangères (FOREIGN KEY).
- Insertion des données : Le jeu d'essai fourni dans l'énoncé a été inséré via des requêtes INSERT INTO afin de remplir la base pour les tests.

3. Requêtes SQL

La dernière partie du TD a permis de manipuler les données pour répondre à des problématiques précises. Plusieurs concepts SQL avancés ont été mis en pratique de manière combinée :

Fonctions d'agrégation : Utilisation de COUNT() pour dénombrer les notes totales.

```
SELECT COUNT(*) FROM NOTER;
```

☐ Profilage [Éditer en ligne] [Éditer]

Options supplémentaires

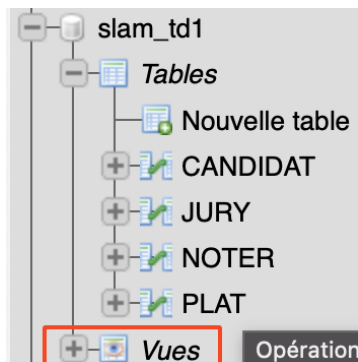
COUNT(*)

45

Création de Vues (CREATE VIEW) : Mise en place de tables virtuelles pour stocker des calculs intermédiaires réutilisables.

- Création d'une vue vue_moyennes pour calculer la moyenne (AVG) des notes groupées par candidat (GROUP BY IdCandidat).

```
1 CREATE VIEW vue_moyennes AS
2 SELECT IdCandidat, AVG(note) AS moyenne_notes
3 FROM NOTER
4 GROUP BY IdCandidat;
```



- Création d'une vue vue_moyennes_jury pour calculer la moyenne des notes données par chaque membre du jury.

```
1 CREATE VIEWS `vue_moyennes_jury` AS
2 SELECT id AVG(note) AS moyenne notes
3 FROM NOTER
4 GROUP by id;
```

idJury	moyenne_notes
1	13.0000
2	13.2222
3	8.3333
4	11.6667
5	17.4444

Jointures (JOIN) et Sous-requêtes : Élaboration de requêtes complexes croisant les tables physiques et les vues. L'objectif était d'extraire des informations textuelles (noms et prénoms) conditionnées par des requêtes de sélection :

- Identification du grand vainqueur en comparant les moyennes avec (SELECT MAX(moyenne_notes)).

```
1 SELECT CANDIDAT.nom, CANDIDAT.prenom, vue_moyennes.moyenne_notes
2 FROM vue_moyennes
3 JOIN CANDIDAT ON vue_moyennes.IdCandidat = CANDIDAT.id
4 WHERE vue_moyennes.moyenne_notes = (SELECT MAX(moyenne_notes) FROM vue_moyennes);
```

nom	prenom	moyenne_notes
touille	rata	13.0667

- Identification du membre du jury le plus sévère avec (SELECT MIN(moyenne_notes)).

```
1 SELECT JURY.nom, JURY.prenom, vue_moyennes_jury.moyenne_notes
2 FROM vue_moyennes_jury
3 JOIN JURY ON vue_moyennes_jury.idJury = JURY.id
4 WHERE vue_moyennes_jury.moyenne_notes = (SELECT MIN(moyenne_notes) FROM
vue_moyennes_jury);
```

nom	prenom	moyenne_notes
Ducasse	Alain	8.3333

- Identification du membre du jury le plus clément avec (SELECT MAX(moyenne_notes))

```
1 SELECT JURY.nom, JURY.prenom, vue_moyennes_jury.moyenne_notes
2 FROM vue_moyennes_jury
3 JOIN JURY ON vue_moyennes_jury.idJury = JURY.id
4 WHERE vue_moyennes_jury.moyenne_notes = (SELECT MAX(moyenne_notes) FROM vue_moyennes_jury);
```

nom	prenom	moyenne_notes
Tout	Bouffe	17.4444

Conclusion

Ce TD a permis de valider la transition d'une modélisation théorique sur papier vers une base de données physique concrète et fonctionnelle. La simplification des associations ternaires a rendu l'architecture de la base évolutive et robuste.

La maîtrise de ces jointures, vues et sous-requêtes constitue une architecture de données fiable, prête à être connectée et exploitée par des procédures algorithmiques ou une future application orientée objet développée en langage Java.

Les associations réflexives

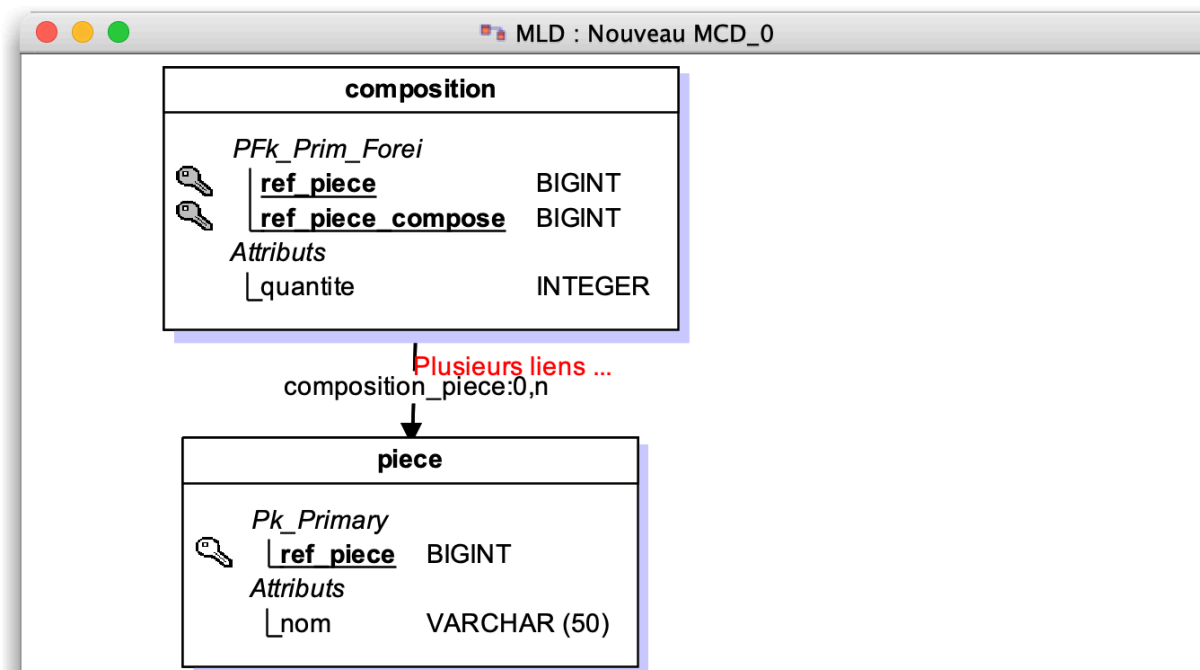
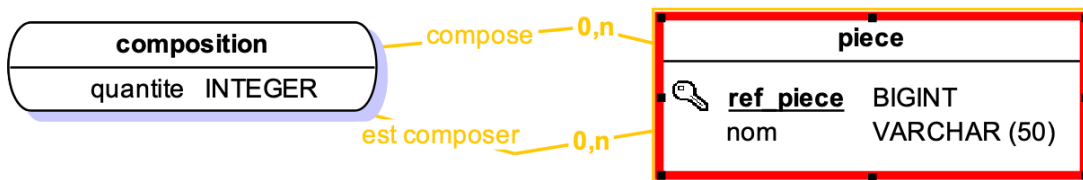
1. Contexte du projet

La société Aile3000 s'occupe de la réparation d'aile des avions. Afin d'optimiser cette activité, la gestion de ses pièces s'effectue dans une base de données. La règle de gestion principale est la suivante : une pièce peut entrer dans la composition de plusieurs pièces (sauf d'elle-même...) et une pièce peut être composée de plusieurs autres pièces. Il s'agit donc de modéliser une nomenclature (ou vue éclatée) avec la notion de quantité.

2. Modélisation Conceptuelle et Logique (Étapes 1 à 3)

Le Modèle Conceptuel de Données (MCD)

Pour répondre à la règle de gestion, nous avons mis en place une association réflexive.



Le Modèle Logique de Données (MLD)

La transformation de l'association réflexive de type (n,n) génère une table intermédiaire contenant les deux clés étrangères pointant vers l'entité PIECE.

3. Création et Alimentation de la Base de Données

Matisse kra
bts sio

Après la création de la base de données SLAM1_TD2 , j'ai pu generer le script sql directement apres la verification de mon mcd le script a été exécuté en s'assurant d'utiliser le moteur de stockage InnoDB pour garantir l'intégrité référentielle

4. Requête et Extraction de données

Étape 10 : Afficher le nom et la quantité des pièces à commander pour une aile

Consigne : On ne détaillera pas les sous pièces.

```
SELECT piece.nom, composition.quantite FROM composition JOIN piece ON composition.Composantid = piece.pieceID WHERE composition.composeId= 'aile';
```

nom	quantite
Aileron d'aile	1
Longeron d'aile	5
rivets à la pelle	100
Train de machin	1

Étape 11 : Afficher le nombre total de pièces à commander pour une aile

Consigne : Sans prendre en compte les sous pièces. Ici, nous utilisons la fonction d'agrégation SUM() pour additionner le volume physique de pièces et non COUNT() qui compterait seulement les types de pièces.

```
1 SELECT SUM(Quantite)
2 FROM COMPOSITION
3 WHERE ComposeId = 'aile';
```

SUM(Quantite)

107

Étape 12 : Afficher le nom et la quantité des pièces à commander pour un "Train de machin"

Consigne : On part du principe qu'on ne connaît pas l'id de la pièce. Pour répondre à cette contrainte, l'utilisation d'une sous-requête dans la clause WHERE s'avère pertinente afin de retrouver dynamiquement l'identifiant à partir du nom de la pièce.

```
1 SELECT piece.nom, composition.quantite
2 FROM composition
3 JOIN piece ON composition.composantId = piece.pieceID
4 WHERE composition.composeId = (
5     SELECT pieceID
6     FROM piece
7     WHERE nom = 'Train de machin'
8 );
```

nom	quantite
Charnière à tout faire	3
rivets à la pelle	8

Conclusion et Bilan du TP

Ce TD m'a permis d'aborder une notion fondamentale et complexe de la modélisation de bases de données : l'association réflexive.

Compétences acquises : L'extraction de données sur une table réflexive m'a permis de pratiquer les jointures complexes et les sous-requêtes, notamment pour retrouver dynamiquement des informations sans connaître l'identifiant de départ (comme exigé à l'étape 12).

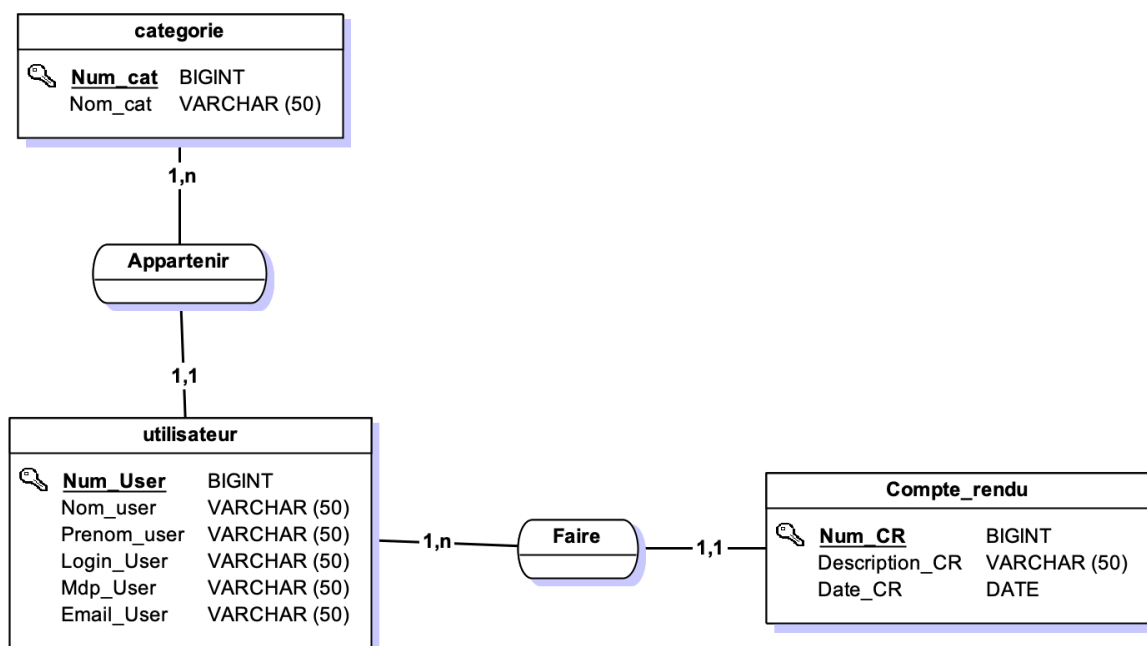
L'héritage

1.Objectif du TD :

l'objectif de ce tp est d'apprendre à manipuler l'héritage sur un mcd c'est à dire au lieu de faire plusieurs entités qui ont les mêmes attributs comment faire pour faire une seule entité qui va regrouper tous les attributs des autres entités

2.création de l'héritage

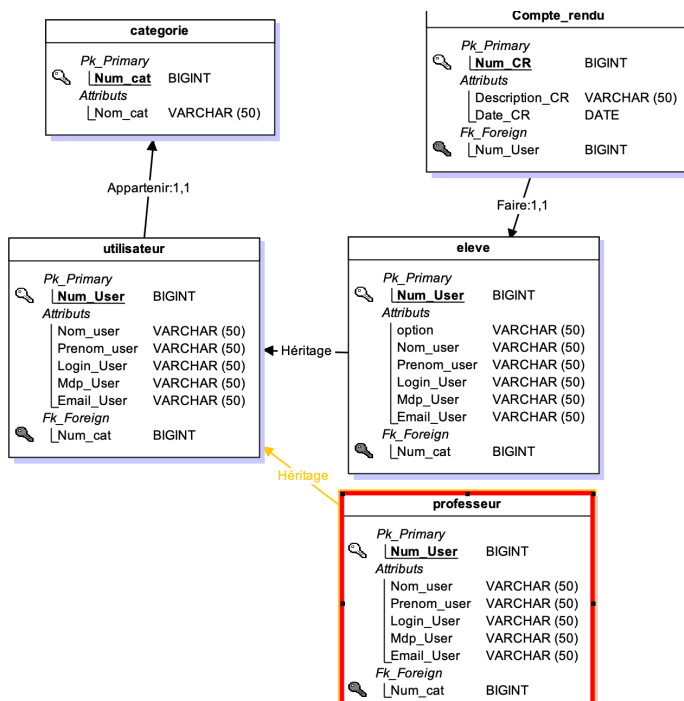
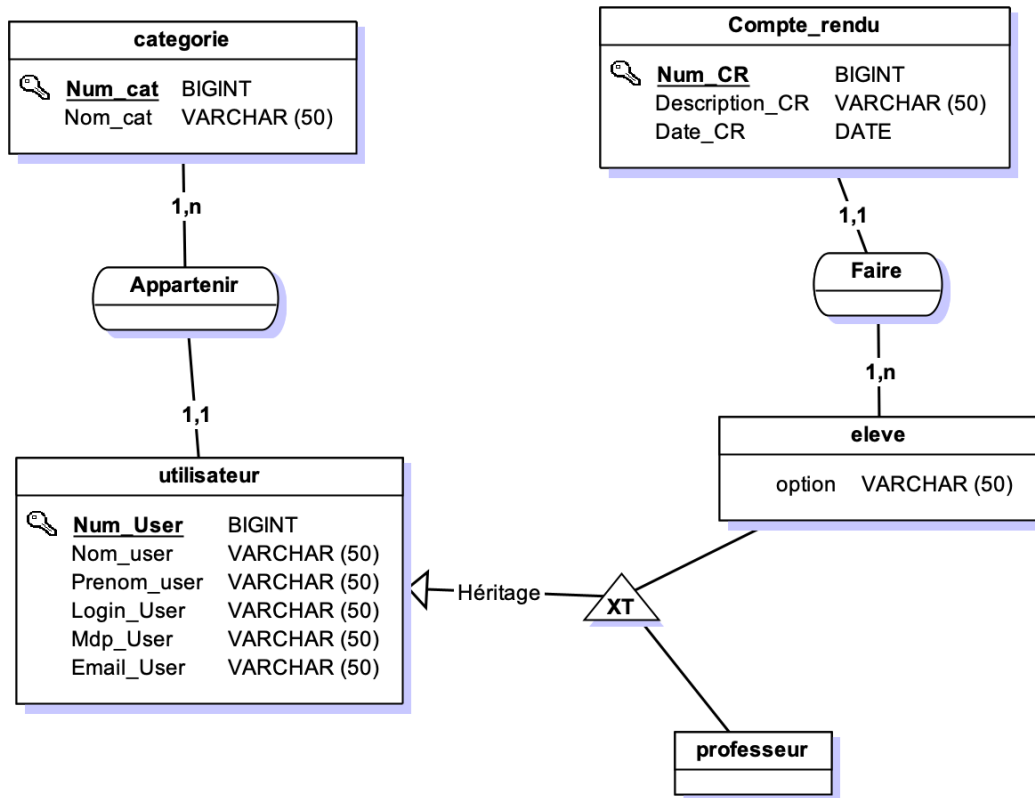
voici le mcd de base



Il nous est demandé de rajouter l'attribut option pour l'élève et que le compte rendu peut être fait seulement par un élève et non un professeur.

Pour répondre à ce besoin on va partir de mcd et transformer l'entité utilisateur en entité mère celle qui contient les attributs communs aux entités élève et professeur puis créer les entités filles entité élève et entité professeur. Dans l'entité élève on ajoute l'attribut option comme il nous est demandé puis on la relie à l'association faire et à l'entité utilisateur. L'entité professeur est seulement reliée à l'entité utilisateur.

Matisse kra
bts sio



conclusion

ce tp m'as permis d'acquérir des compétence approfondie à la réalisation de la modélisation d'un mcd en manipulant les entité mère et fille

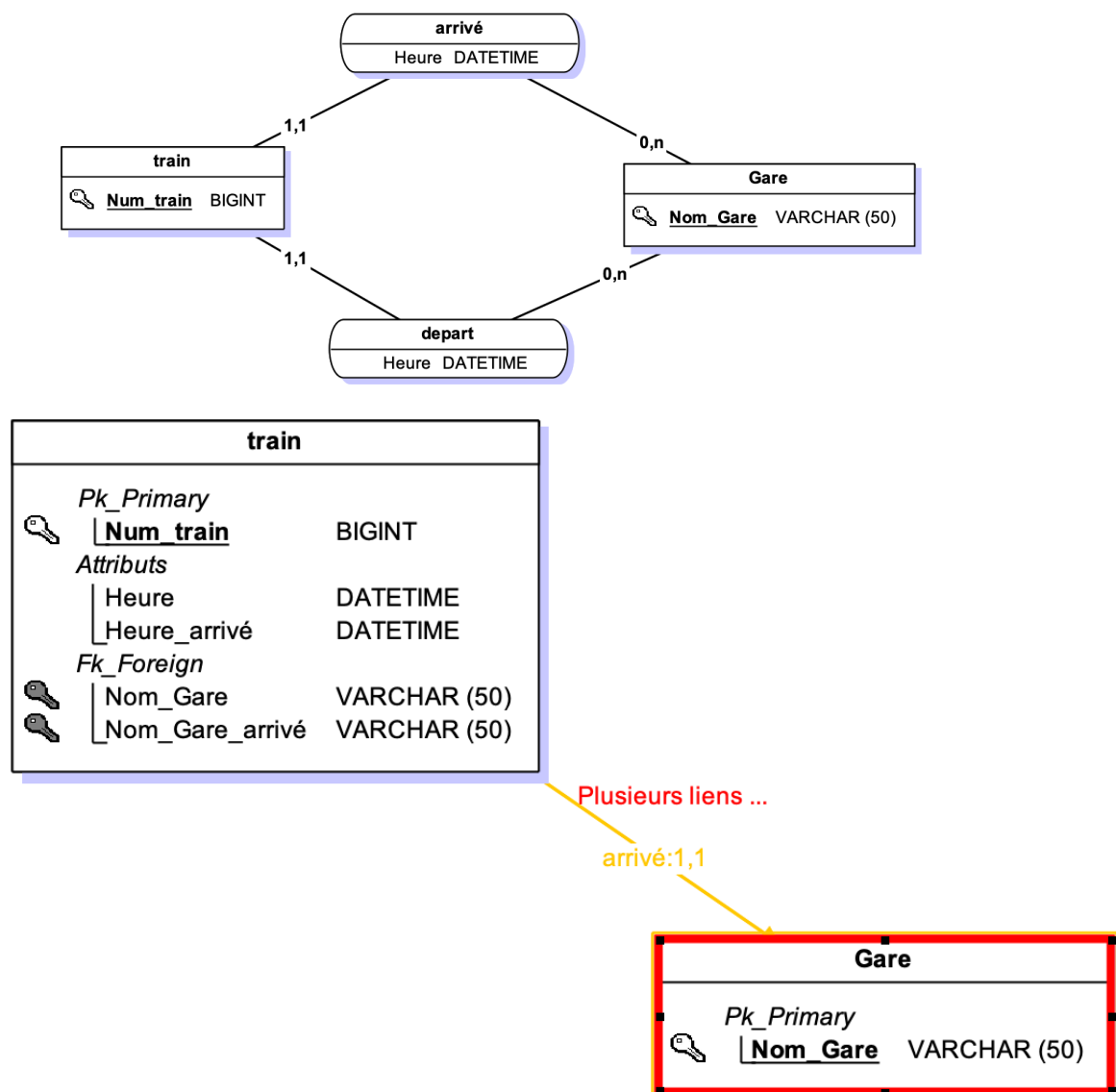
relation multiple

1.Objectifs du TP

L'objectif était de modéliser une base de données capable de gérer des trains, des lignes commerciales et des horaires de passage dans différentes gares. Le TP a évolué d'une relation simple (Départ/Arrivée) vers une gestion complète des gares desservies (étapes intermédiaires).

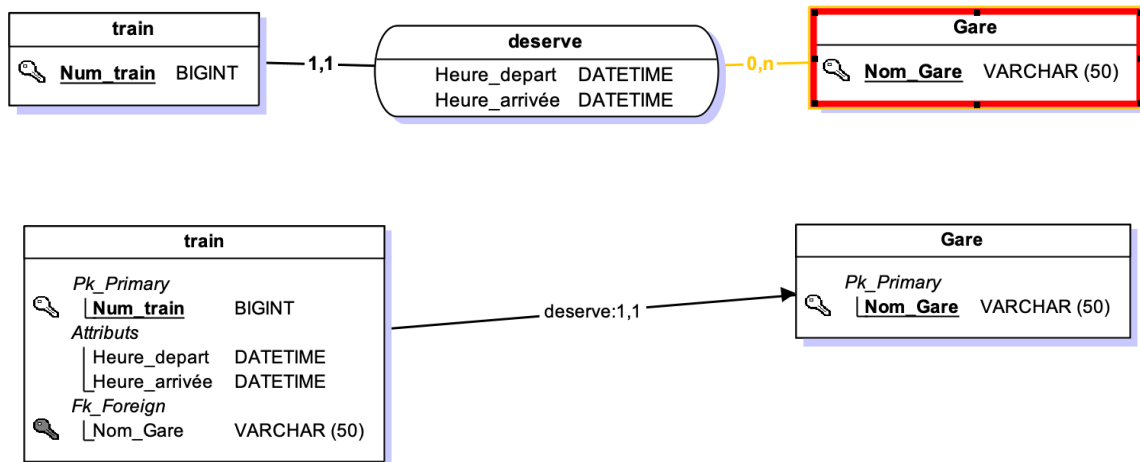
2.Modélisation Conceptuelle (MCD)

un premier modèle de mcd et mld a été créé avec deux association l'association arrivée et l'association départ



Matisse kra
bts sio

ensuite on s'est aperçu que la sncf souhaite enregistrer pour chaque train les gares desservies, ainsi que l'horaire d'arrivée de départ dans ces gares. nouveau mcd et mld ont été réalisés



3. base de donnée

dans la base de donnée j'ai implémenter deux tables de plus qui n'était pas demander dans la réalisation du mcd mais que j'ai jugé nécessaire pour répondre au problème final j'ai créé les tables :

- ligne qui comprend la ligne empruntée par le train
- desservir les gare desservi par le train

j'ai ensuite implémenter dans ma base ces deux train

étape 7 : Insérer dans votre base de données le train n°242:

- ligne n°2 - vert : Simplon-Orient-Express
- départ de Paris le 10/03/2020 à 8h30
- arrivée à Istanbul le 15/03/2020 à 16h45
- tarif : 4000€
- gares desservies :
 - Lausanne : le 10/03/2020 à 16h15
 - Milan : le 11/03/2020 à 9h45
 - Venise : le 12/03/2020 à 8h
 - Belgrade : le 13/03/2020 à 19h55
 - Sofia : le 14/03/2020 à 16h

étape 8 : Insérer dans votre base de données le train n°422:

- ligne n°4 - violet : Venise-Simplon-Orient-Express
- départ de Calais le 10/03/2020 à 9h00
- arrivée à Venise le 12/03/2020 à 20h30
- tarif : 2360 €
- gares desservies :
 - Paris : le 10/03/2020 à 12h
 - Lausanne : le 11/03/2020 à 8h
 - Milan : le 11/03/2020 à 21h

Matisse kra
bts sio

script sql :

```
1 INSERT IGNORE INTO Gare (Nom_Gare) VALUES
2 ('Paris'), ('Istanbul'), ('Lausanne'), ('Milan'),
3 ('Venise'), ('Belgrade'), ('Sofia'), ('Calais');
4
5 INSERT INTO train (Num_train, num_ligne, couleur, tarif)
6 VALUES (242, 2, 'vert', 4000.00);
7
8 INSERT INTO desservir (Num_train, Nom_Gare, Heure_arrivee, Heure_depart) VALUES
9 (242, 'Paris', NULL, '2020-03-10 08:30:00'),
10 (242, 'Lausanne', '2020-03-10 16:15:00', NULL),
11 (242, 'Milan', '2020-03-11 09:45:00', NULL),
12 (242, 'Venise', '2020-03-12 08:00:00', NULL),
13 (242, 'Belgrade', '2020-03-13 19:55:00', NULL),
14 (242, 'Sofia', '2020-03-14 16:00:00', NULL),
15 (242, 'Istanbul', '2020-03-15 16:45:00', NULL);
16
17
18 INSERT INTO train (Num_train, num_ligne, couleur, tarif)
19 VALUES (422, 4, 'violet', 2360.00);
20
21
22 INSERT INTO desservir (Num_train, Nom_Gare, Heure_arrivee, Heure_depart) VALUES
23 (422, 'Calais', NULL, '2020-03-10 09:00:00'),
24 (422, 'Paris', '2020-03-10 12:00:00', NULL),
25 (422, 'Lausanne', '2020-03-11 08:00:00', NULL),
26 (422, 'Milan', '2020-03-11 21:00:00', NULL),
27 (422, 'Venise', '2020-03-12 20:30:00', NULL);
```

			Num_train	Nom_Gare	Heure_arrivee	Heure_depart
☐	 Éditer	 Copier	 Supprimer	242	Belgrade	2020-03-13 19:55:00 NULL
☐	 Éditer	 Copier	 Supprimer	242	Istanbul	2020-03-15 16:45:00 NULL
☐	 Éditer	 Copier	 Supprimer	242	Lausanne	2020-03-10 16:15:00 NULL
☐	 Éditer	 Copier	 Supprimer	242	Milan	2020-03-11 09:45:00 NULL
☐	 Éditer	 Copier	 Supprimer	242	Paris	NULL 2020-03-10 08:30:00
☐	 Éditer	 Copier	 Supprimer	242	Sofia	2020-03-14 16:00:00 NULL
☐	 Éditer	 Copier	 Supprimer	242	Venise	2020-03-12 08:00:00 NULL
☐	 Éditer	 Copier	 Supprimer	422	Calais	NULL 2020-03-10 09:00:00
☐	 Éditer	 Copier	 Supprimer	422	Lausanne	2020-03-11 08:00:00 NULL
☐	 Éditer	 Copier	 Supprimer	422	Milan	2020-03-11 21:00:00 NULL
☐	 Éditer	 Copier	 Supprimer	422	Paris	2020-03-10 12:00:00 NULL
☐	 Éditer	 Copier	 Supprimer	422	Venise	2020-03-12 20:30:00 NULL

Conclusion

Ce TP a permis de mettre en pratique la gestion des relations plusieurs-à-plusieurs.
L'utilisation d'une table de jointure (desservir) s'est avérée indispensable pour gérer la

Matisse kra
bts sio

flexibilité des trajets ferroviaires, contrairement à une simple association de un-à-un qui aurait limité le train à seulement deux gares.

la gestion des jointures

1.Objectifs du TP

L'objectif était de modéliser une base de données capable de référencer les voitures des clients en utilisant des requêtes de jointure.

2.Modélisation Conceptuelle (MCD)

Le modèle conceptuel de données (MCD) proposé initialement présentait une association ternaire nommée POSSEDER reliant simultanément Personne, Voiture et Couleur.

Critiques formulées :

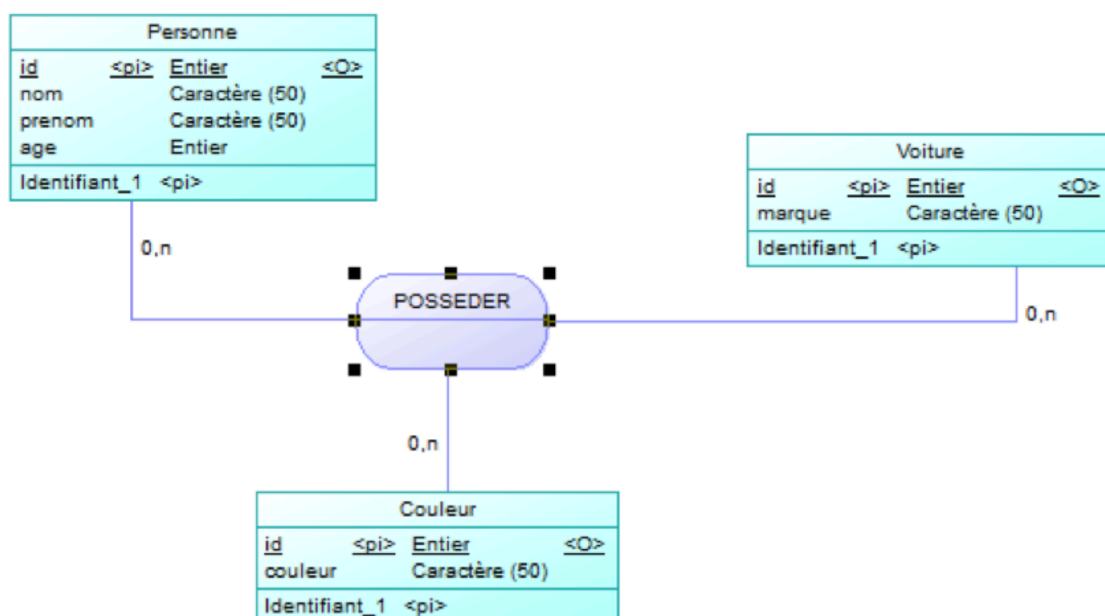
Dans la réalité, la couleur est une caractéristique de la voiture et non de la personne. Lier les trois entités impose qu'une voiture n'a de couleur que si elle est possédée.

Corrections apportées :

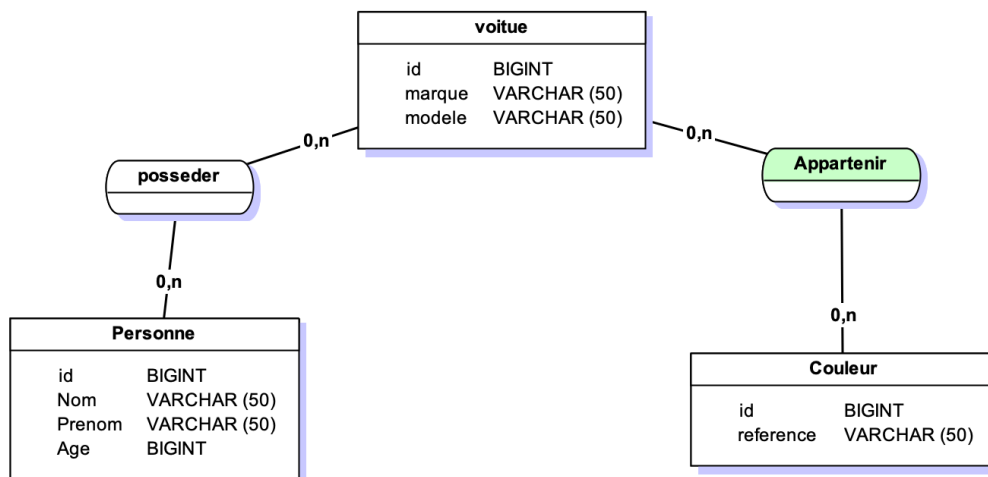
Décomposition de l'association ternaire en deux associations binaires :

1. posseder (entre Personne et Voiture).
2. Appartenir (entre Voiture et Couleur).

ancien mcd



nouveau mcd corriges



3. Implémentation de la Base de Données

La base de données nommée slam_td7 a été générée avec les structures suivantes:

- Tables : Personne, voiture, Couleur.
- Tables de liaison : posseder (clés id_p, id_v) et Appartenir (clés id_c, id_v).

Les données insérées respectent l'énoncé, permettant de tester des cas de figures variés (personnes sans voiture, voitures à plusieurs couleurs).

4. Travaux de Requêtage (SQL)

A. Jointures Internes (INNER JOIN)

L'objectif était d'afficher les correspondances strictes.

question 3 afficher la liste des personne et leur voiture

```

1 SELECT Personne.Nom, Personne.Prenom, voiture.marque
2 FROM Personne, voiture, posseder
3 WHERE Personne.id_P = posseder.id_p
4 AND voiture.id_v = posseder.id_v;
  
```

Nom	Prenom	marque
rose	fabienne	Mercedes
simond	victor	Mercedes
simond	victor	BMW

question 4 reprendre la question q3 en faisant une jointure "inner join"

```

1 SELECT Personne.Nom, Personne.Prenom, voiture.marque
2 FROM Personne
3 INNER JOIN posseder ON Personne.id_P = posseder.id_p
4 INNER JOIN voiture ON posseder.id_v = voiture.id_v;
  
```

Nom	Prenom	marque
rose	fabienne	Mercedes
simond	victor	Mercedes
simond	victor	BMW

Question 6 : Affichage des personnes, leurs voitures et la couleur associée.

```

1 SELECT Personne.Nom, Personne.Prenom, voiture.marque, Couleur.reference
2 FROM Personne
3 INNER JOIN posseder ON Personne.id_P = posseder.id_p
4 INNER JOIN voiture ON posseder.id_v = voiture.id_v
5 INNER JOIN Appartenir ON voiture.id_v = Appartenir.id_v
6 INNER JOIN Couleur ON Appartenir.id_c = Couleur.id_c;

```

Nom	Prenom	marque	reference
rose	fabienne	Mercedes	rouge
rose	fabienne	Mercedes	bleu
simond	victor	Mercedes	rouge
simond	victor	Mercedes	bleu
simond	victor	BMW	jaune
simond	victor	BMW	bleu

Note technique : Suite à la séparation des tables, la requête nécessite désormais quatre jointures successives pour lier les quatre tables via leurs clés étrangères respectives.

Question 7 : Comparaison des syntaxes de jointure

(L'objectif de cette question était de comparer le résultat obtenu avec un INNER JOIN (Question 6) et celui obtenu avec une jointure effectuée via la clause WHERE (syntaxe "produit cartésien filtré").

```

1 SELECT Personne.Nom, Personne.Prenom, voiture.marque, Couleur.reference
2 FROM Personne, posseder, voiture, Appartenir, Couleur
3 WHERE Personne.id_P = posseder.id_p
4 AND posseder.id_v = voiture.id_v
5 AND voiture.id_v = Appartenir.id_v
6 AND Appartenir.id_c = Couleur.id_c;

```

Matisse kra
bts sio

Nom	Prenom	marque	reference
rose	fabienne	Mercedes	rouge
rose	fabienne	Mercedes	bleu
simond	victor	Mercedes	rouge
simond	victor	Mercedes	bleu
simond	victor	BMW	jaune
simond	victor	BMW	bleu

Résultat : Le résultat est strictement identique en termes de lignes et de colonnes affichées. Les deux méthodes retournent l'intersection exacte des données liées.

Lisibilité : La syntaxe avec INNER JOIN est jugée plus claire car elle sépare la structure de la base (comment les tables sont liées dans le FROM) de la logique métier (le filtrage des données dans le WHERE).

B. Jointures Externes (LEFT / RIGHT JOIN)

Ces jointures ont été utilisées pour traiter les données sans correspondance.

question 5 afficher la liste des personnes et leur voiture même ceux qui n'en possèdent pas

```
1 SELECT Personne.Nom, Personne.Prenom, voiture.marque
2 FROM Personne
3 LEFT JOIN posseder ON Personne.id_P = posseder.id_p
4 LEFT JOIN voiture ON posseder.id_v = voiture.id_v;
```

Nom	Prenom	marque
dupont	jean	NULL
rose	fabienne	Mercedes
simond	victor	Mercedes
simond	victor	BMW
vic	hugo	NULL

question 8 Affichage de toutes les personnes, y compris celles n'ayant pas de voiture ainsi que leur couleur

```
1 SELECT Personne.Nom, Personne.Prenom, voiture.marque, Couleur.reference
2 FROM Personne
3 left JOIN posseder on Personne.id_P = posseder.id_p
4 LEFT JOIN voiture on posseder.id_v = voiture.id_v
5 LEFT JOIN Appartenir ON voiture.id_v = Appartenir.id_v
6 LEFT JOIN Couleur ON Appartenir.id_c = Couleur.id_c;
```

Nom	Prenom	marque	reference
dupont	jean	NULL	NULL
rose	fabienne	Mercedes	rouge
rose	fabienne	Mercedes	bleu
simond	victor	Mercedes	rouge
simond	victor	Mercedes	bleu
simond	victor	BMW	jaune
simond	victor	BMW	bleu
vic	hugo	NULL	NULL

question 9 Affichage de toutes les voitures, y compris celles non possédées

```
SELECT Personne.Nom, Personne.Prenom, voiture.marque, Couleur.reference
FROM voiture
LEFT JOIN posseder ON voiture.id_v = posseder.id_v
LEFT JOIN Personne ON posseder.id_p = Personne.id_P
LEFT JOIN Appartenir ON voiture.id_v = Appartenir.id_v
LEFT JOIN Couleur ON Appartenir.id_c = Couleur.id_c;
```

Nom	Prenom	marque	reference
rose	fabienne	Mercedes	rouge
rose	fabienne	Mercedes	bleu
simond	victor	Mercedes	rouge
simond	victor	Mercedes	bleu
simond	victor	BMW	jaune
simond	victor	BMW	bleu
NULL	NULL	Renault	NULL
NULL	NULL	Ferrari	NULL

C. Filtrage Spécifique (Question 10)

Pour isoler les voitures sans propriétaire, nous avons combiné une jointure externe avec un filtre de valeur nulle :

```
1 SELECT voiture.marque, Personne.Nom
2 FROM voiture
3 LEFT JOIN posseder ON voiture.id_v = posseder.id_v
4 LEFT JOIN Personne ON posseder.id_p = Personne.id_P
5 WHERE Personne.id_P IS NULL;
```

marque	Nom
Renault	NULL
Ferrari	NULL

Cette méthode permet d'identifier précisément les données orphelines dans la relation de possession.

5. Conclusion

Ce TP a permis de valider la maîtrise des différents types de jointures SQL. La modification du MCD initial vers un modèle normalisé a complexifié l'écriture des requêtes (plus de tables à joindre) mais a rendu la base de données beaucoup plus évolutive et fidèle aux règles de gestion réelles.

UNION & VUES

1. Présentation du Contexte

Dans le cadre de la gestion des stages, l'objectif était d'extraire et de centraliser les contacts des entreprises, des tuteurs et des élèves au sein d'une structure unique pour faciliter la communication des prochaines promotions.

2. Problématique Technique

Comment regrouper des informations issues de tables ayant des structures différentes (ENTREPRISE, TUTEUR, UTILISATEUR) dans un format d'affichage standardisé (Type, Nom, Tel, Email) ?

3. Compétences Mises en Œuvre

- **question 1 et 2 Algèbre relationnelle** : Utilisation de l'opérateur UNION pour la fusion verticale de jeux de données.

```
1 SELECT Nom, tel
2 FROM Entreprise
3 UNION
4 SELECT Nom, tel
5 FROM Tuteur;
```

```
1 SELECT Nom, tel ,ADRESSE
2 FROM Entreprise
3 UNION
4 SELECT Nom, tel ,MAIL
5 FROM Tuteur;
```

- **question 3 Définition de données** : Création de vues SQL (CREATE VIEW) pour pérenniser les requêtes complexes.

```
1 CREATE VIEW Renseignement AS
2 SELECT 'Ent' AS Type, Nom, tel, adresse AS email
3 FROM Entreprise
4 UNION
5 SELECT 'Tut' AS Type, Nom, tel, mail
6 FROM Tuteur;
```

Type	Nom	tel	email
Ent	CyberTech Solutions	0140203040	12 rue de la Paix, Paris
Ent	EcoLogic SAS	0250607080	5 Avenue des Fleurs, Lyon
Tut	Gauthier	0610203040	m.gauthier@cybertech.fr
Tut	Vidal	0780901020	lucie.vidal@ecologic.com

- **question 4 modification de vues SQL et Jointures** : Association de tables liées par héritage (ELEVE et UTILISATEUR) pour récupérer des informations nominatives. Utilisation de AS pour harmoniser les noms de colonnes et identifier la provenance des données (Type "Ent", "Tut" ou "El").

```
1 CREATE OR REPLACE VIEW Renseignement AS
2 -- Partie Entreprises
3 SELECT 'Ent' AS Type, Nom, tel, adresse AS email
4 FROM Entreprise
5 UNION
6 -- Partie Tuteurs
7 SELECT 'Tut' AS Type, Nom, tel, mail
8 FROM Tuteur
9 UNION
10 -- Partie Elèves (Jointure nécessaire)
11 SELECT 'El' AS Type, Utilisateur.Nom, Utilisateur.tel, Utilisateur.mail
12 FROM Eleve
13 INNER JOIN Utilisateur ON Eleve.Num = Utilisateur.Num;
```

Type	Nom	tel	email
Ent	CyberTech Solutions	0140203040	12 rue de la Paix, Paris
Ent	EcoLogic SAS	0250607080	5 Avenue des Fleurs, Lyon
Tut	Gauthier	0610203040	m.gauthier@cybertech.fr
Tut	Vidal	0780901020	lucie.vidal@ecologic.com
El	Durand	NULL	thomas.durand@bts.edu
El	Petit	NULL	chloe.petit@bts.edu

5. Schéma de Données (MCD/MLD)

Le projet repose sur une architecture où la table `UTILISATEUR` centralise les informations de connexion et d'identité, tandis que les tables `ELEVE` et `PROF` en héritent.

6. conclusion

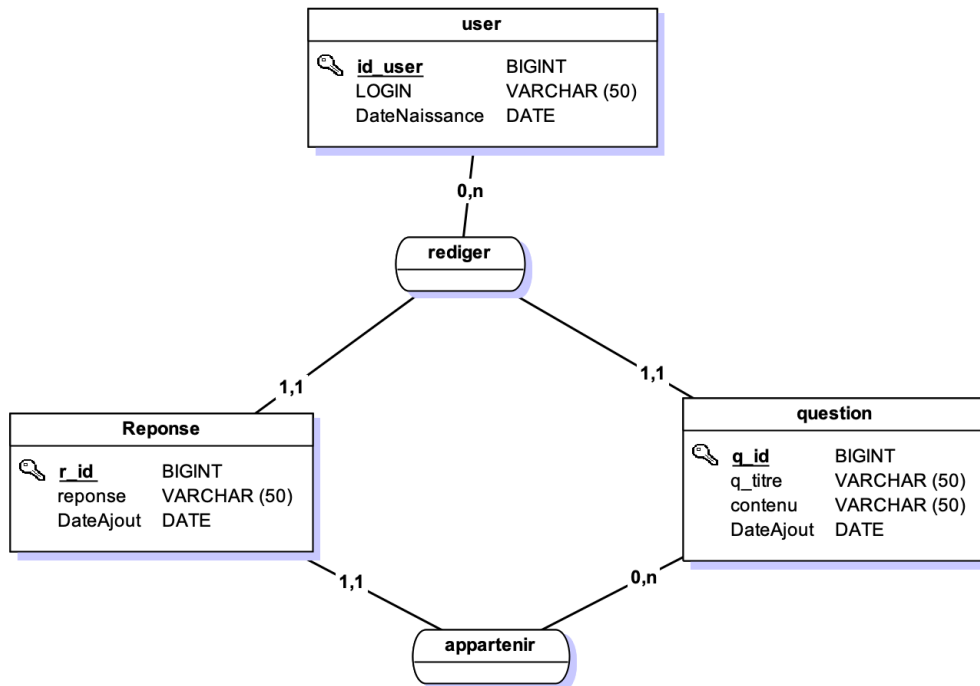
Ce TP m'a permis de comprendre la différence entre une jointure (qui étend une ligne horizontalement) et une union (qui ajoute des lignes verticalement). L'utilisation de `UNION` impose une rigueur stricte sur l'ordre et le type des colonnes sélectionnées.

Gestion d'un Forum

1. Analyse et Modélisation (MCD) question 1 et 8

Le but était de concevoir la structure d'un forum permettant de poser des questions et d'y répondre.

- Entités créées : question, Reponse et utilisateur.
- Relations :
 - Une question peut recevoir plusieurs réponses (0,n), mais une réponse appartient à une seule question (1,1).
 - Un utilisateur peut poster plusieurs questions et plusieurs réponses (0,n).



2. Mise en œuvre de la Base de Données (SQL) question 2 et 3, 9 et 10

La base de données slam_td9 a été générée et configurée sous phpMyAdmin.

Table	Action	Lignes	Type	Interclassement	Taille	Perte
☐ question	Parcourir Structure Rechercher Insérer Vider Supprimer	4	InnoDB	utf8mb4_general_ci	32,0 kio	-
☐ Reponse	Parcourir Structure Rechercher Insérer Vider Supprimer	4	InnoDB	utf8mb4_general_ci	48,0 kio	-
☐ utilisateur	Parcourir Structure Rechercher Insérer Vider Supprimer	3	InnoDB	utf8mb4_general_ci	16,0 kio	-
3 tables Somme		11	InnoDB	utf8mb4_general_ci	96,0 kio	0 o

Le jeu de données de test comprend:

- **3 Utilisateurs** : Bob, Steeve et Walee.
- **4 Questions** techniques (réparation PC, changement de pneu, etc.).
- **4 Réponses** distribuées entre les différents sujets.

	id_user	LOGIN	DateNaissance
☐ Éditer Copier Supprimer	1	bob	1980-01-01 00:00:00
☐ Éditer Copier Supprimer	2	steeve	1970-01-01 00:00:00
☐ Éditer Copier Supprimer	3	walee	1990-01-01 00:00:00

3. Manipulation et Extraction de données question 4 a 7

Nous avons réalisé plusieurs requêtes complexes pour analyser l'activité du forum :

- Sélection par date question 4 et 5 : Extraction des questions liées à des réponses entre deux dates précises.

question 4 : trouve la requête permettant de connaître la question liée à la dernière reponse ajouté sur l'application

```
1 SELECT q.q_titre, r.reponse
2 FROM question q
3 JOIN Reponse r ON q.q_id = r.q_id
4 WHERE r.DateAjout = (SELECT MAX(DateAjout) FROM Reponse);
```

q_titre	reponse
Que faire si un appareil est cassé?	Bonjour. Ca dépend de vous, de votre budget et de ...

question 5 : trouve la requête permettant d'obtenir toute les reponse comprise entre 2 dates

```
1 SELECT q.q_titre, r.reponse
2 FROM question q
3 JOIN Reponse r ON q.q_id = r.q_id
4 WHERE r.DateAjout BETWEEN '2013-03-27' AND '2013-05-20';
```

q_titre	reponse
Comment réparer un ordinateur?	Bonjour. Pouvez-vous expliquer ce qui ne fonctionn...
Comment réparer un ordinateur?	Bonsoir, le plus simple consiste à faire appel à u...
Comment changer un pneu?	Des conseils son disponible sur internet sur ce su...

- Analyse de contenu question 6 : Identification des questions n'ayant reçu aucune réponse (via un LEFT/RIGHT JOIN ou NOT IN).

question 6 : trouve la requête permettant d'obtenir les questions sans reponse

- Statistiques de participation question 7 et 13 et 14 :
 - Recherche de la question la plus populaire (plus grand nombre de réponses).
 - Identification de l'utilisateur le plus actif en additionnant ses questions et ses réponses postées.

question 7 trouve la requête permettant d'obtenir la question qui a obtenu le plus de reponse

```
1 SELECT q.q_titre
2 FROM Reponse r
3 RIGHT JOIN question q ON r.q_id = q.q_id
4 WHERE r.r_id IS NULL;
```

q_titre

Comment faire nettoyer un clavier d'ordinateur?

question 13 trouve la requête permettant d'obtenir l'utilisateur qui a poster le plus de reponse

```
1 SELECT u.LOGIN, COUNT(r.r_id) AS nb_reponses
2 FROM utilisateur u
3 JOIN Reponse r ON u.id_user = r.id_user
4 GROUP BY u.id_user
5 ORDER BY nb_reponses DESC
6 LIMIT 1;
```

LOGIN	nb_reponses
-------	-------------

steeve	3
--------	---

question 14 trouve la requête permettant d'obtenir l'utilisateur le plus actif

```
1 SELECT u.LOGIN,
2       (SELECT COUNT(*) FROM question q WHERE q.id_user = u.id_user) +
3       (SELECT COUNT(*) FROM Reponse r WHERE r.id_user = u.id_user) AS total_activite
4 FROM utilisateur u
5 ORDER BY total_activite DESC
6 LIMIT 1;
```

LOGIN	total_activite	▼ 1
-------	----------------	-----

steeve	3
--------	---

walee	3
-------	---

bob	2
-----	---

Conclusion technique

Le projet a permis de valider la maîtrise des jointures, des fonctions d'agrégation (COUNT, MAX) et surtout des sous-requêtes, essentielles pour croiser des informations provenant de plusieurs tables tout en respectant les contraintes d'intégrité de la base.